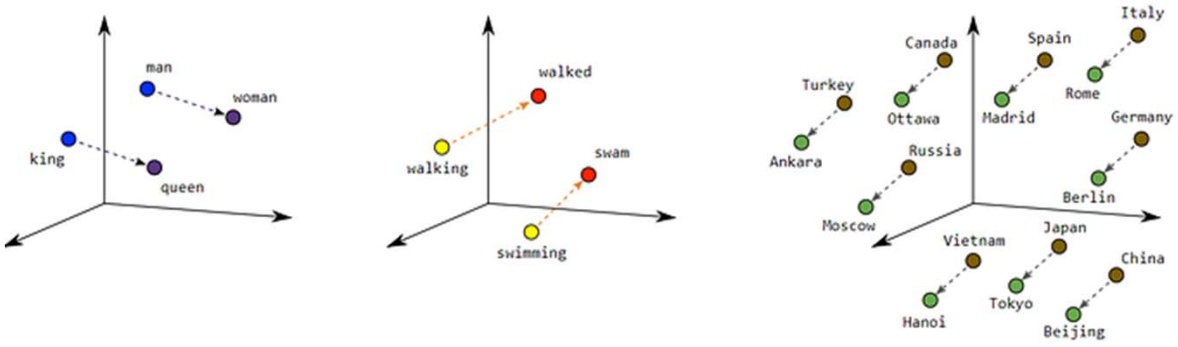
NLP

Lecture 8

Word Vectors and Vector Semantics

Sine Zambach, Department of Digitalization, Copenhagen Business School



Male-Female Verb Tense Country-Capital

Based on ***CS224n: Natural Language Processing with Deep Learning Stanford / Winter 2019*** http://web.stanford.edu/class/cs224n/index.html#schedule

Agenda for today

Vector Semantics

- Meaning (semantics) and context
- Words and Vectors
- Cosine Similarity

Word2Vec

- Skip-Gram model

Recap – what AI systems do you know?

- Modelled, **supervised**
- Data based **supervised** (classification using log reg, random forrest, ANNs…)
- Data based **unsupervised** (clustering, topic modelling)
- Data based **self-supervised** (language based supervision)

Symbolic AI

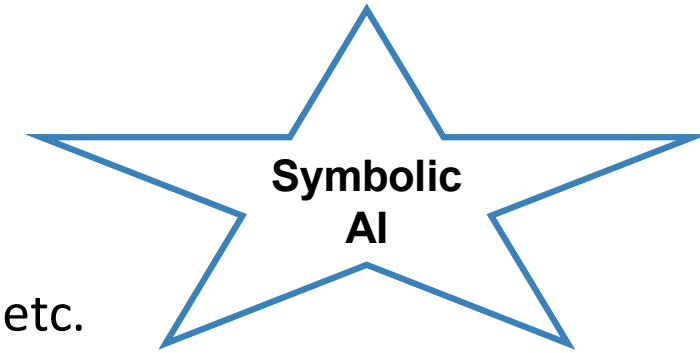
Sub-symbolic AI

What is meaning?

So far in the course?

In for instance large language models?

How do we represent the meaning of a word?



Definition: **meaning** (Webster dictionary)

- the *idea* that is represented by a word, phrase, etc.
- the *idea* that a person wants to express by using words, signs, etc.
- the *idea* that is expressed in a work of writing, art, etc.

Commonest linguistic way of thinking of meaning:

signifier (symbol) ⟺ signified (idea or thing)

= denotational **semantics**

How do we have usable meaning in a computer?

Old (?) solution: Use ontologies e.g. WordNet, a thesaurus containing lists of **synonym sets** and **hypernyms** ("is a" relationships).

e.g. synonym sets containing "good":

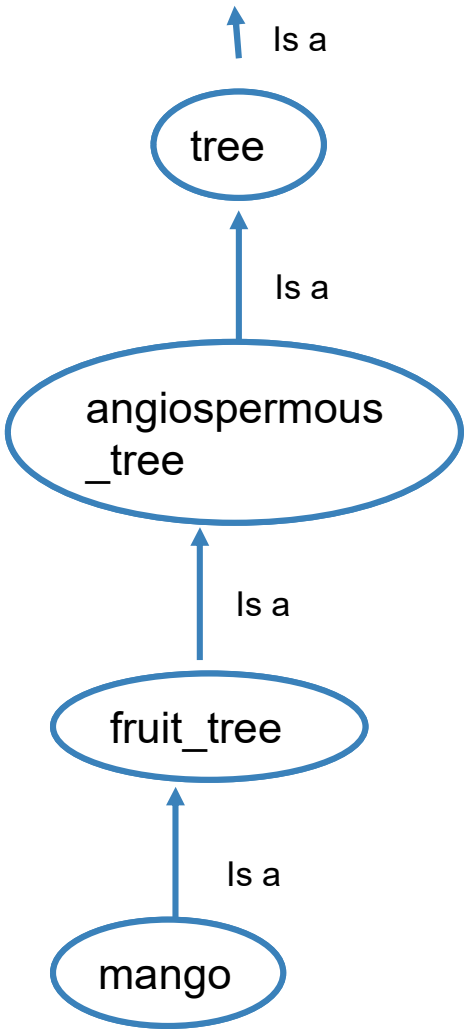
```
from nltk.corpus import wordnet as wn
poses = { 'n':'noun', 'v':'verb', 's':'adj (s)',
'a':'adj', 'r':'adv'}
for synset in wn.synsets("good"):
    print("{}: {}".format(poses[synset.pos()],
            ", ".join([l.name() for l in
synset.lemmas()])))
```

```
noun: good
noun: good, goodness
noun: good, goodness
noun: commodity, trade_good, good
adj: good
adj (sat): full, good
adj: good
adj (sat): estimable, good, honorable, respectable  adj
(sat): beneficial, good
adj (sat): good
adj (sat): good, just, upright
…
adverb: well, good
adverb: thoroughly, soundly, good
```

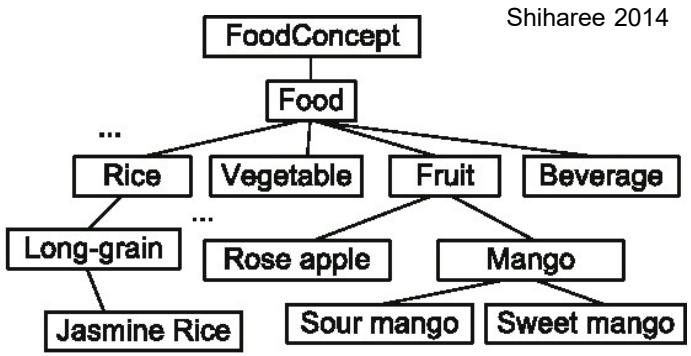
e.g. hypernyms of "mango":

```
from nltk.corpus import wordnet as wn
panda = wn.synset("mango.n.01")
hyper = lambda s: s.hypernyms()
list(panda.closure(hyper))
```

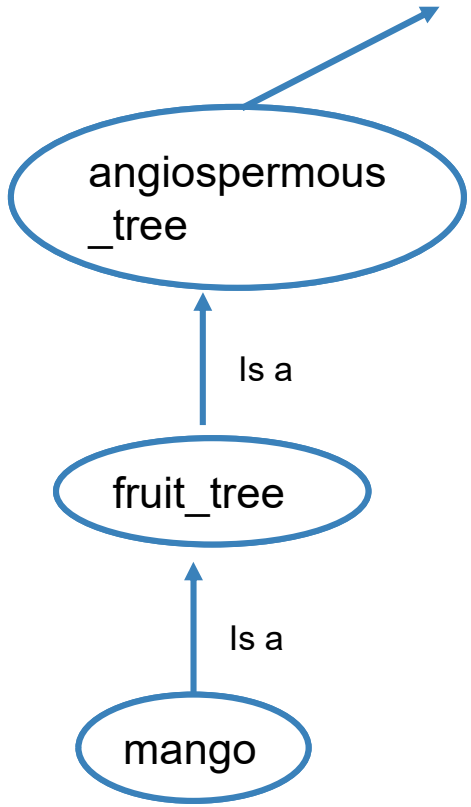
```
[Synset('fruit_tree.n.01'),
 Synset('angiospermous_tree.n.01'),
 Synset('tree.n.01'),
 Synset('woody_plant.n.01'),
 Synset('vascular_plant.n.01'),
 Synset('plant.n.02'),
 Synset('organism.n.01'),
 Synset('living_thing.n.01'),
 Synset('whole.n.02'),
 Synset('object.n.01'),
 Synset('physical_entity.n.01'),
 Synset('entity.n.01')]
```



Problems with resources like WordNet



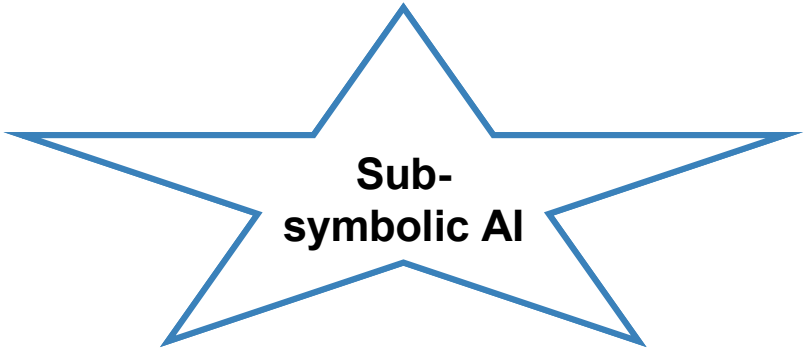
- Great as a resource but missing nuance
  - e.g. "proficient" is listed as a synonym for "good". This is only correct in some contexts.
- Missing new meanings of words
  - e.g., wicked, badass, nifty, wizard, genius, ninja, bombest
  - Impossible to keep up-to-date!
- Requires human labor to create and curate
- Subjective
- Can't compute accurate word similarity



Let's define words by their usages

In particular, words are defined by their environments (the words around them)

Zellig Harris (1954): **If A and B have almost identical environments we say that they are synonyms**.



SO: Representing words as discrete symbols based on context

In traditional NLP, we regard words as discrete symbols:
hotel, conference, motel – a localist representation

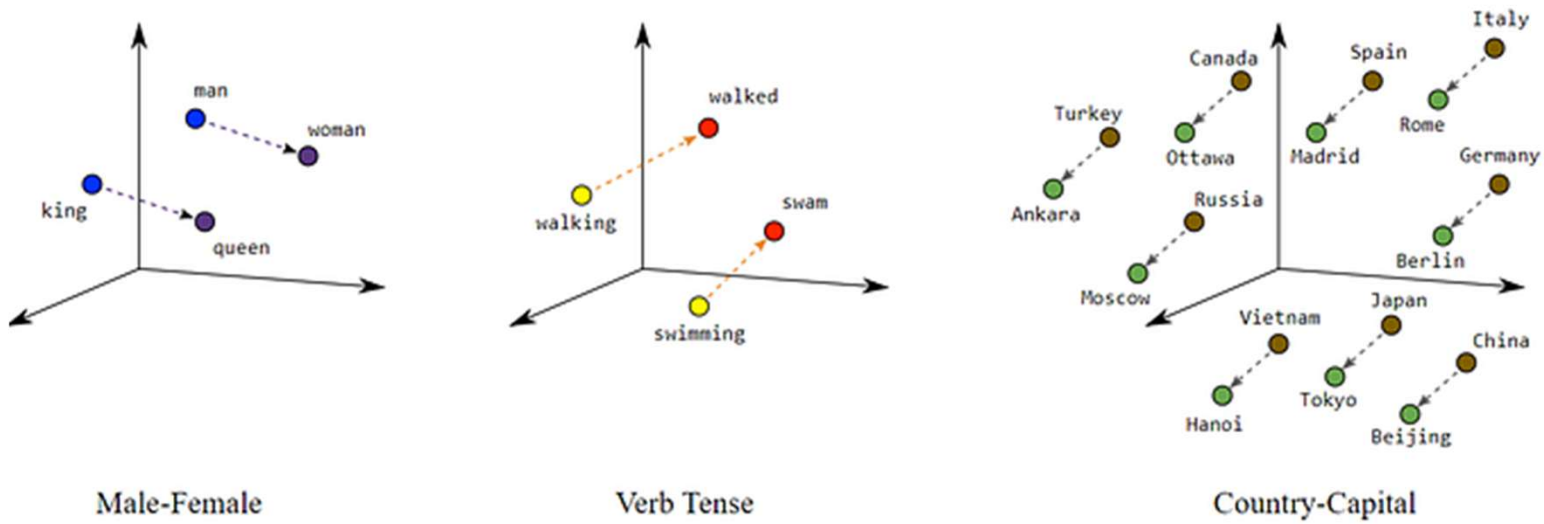
Means one 1, the rest 0s

Words can be represented by **one-hot** vectors:

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]
hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

Vector dimension = number of words in vocabulary (e.g., 500,000)
SPARSE vectors

Source for Illustration



Vector Semantics

We'll introduce 2 kinds of embeddings

Word-context matrix/Co-occurence matrix

- A common baseline model
- Sparse vectors
- Words are represented by a simple function of the counts of nearby words

Word2vec (2013)

- **Dense** vectors
- Representation is created by training a classifier to distinguish nearby and far-away words
- First step towards LLMs

Term-document matrix

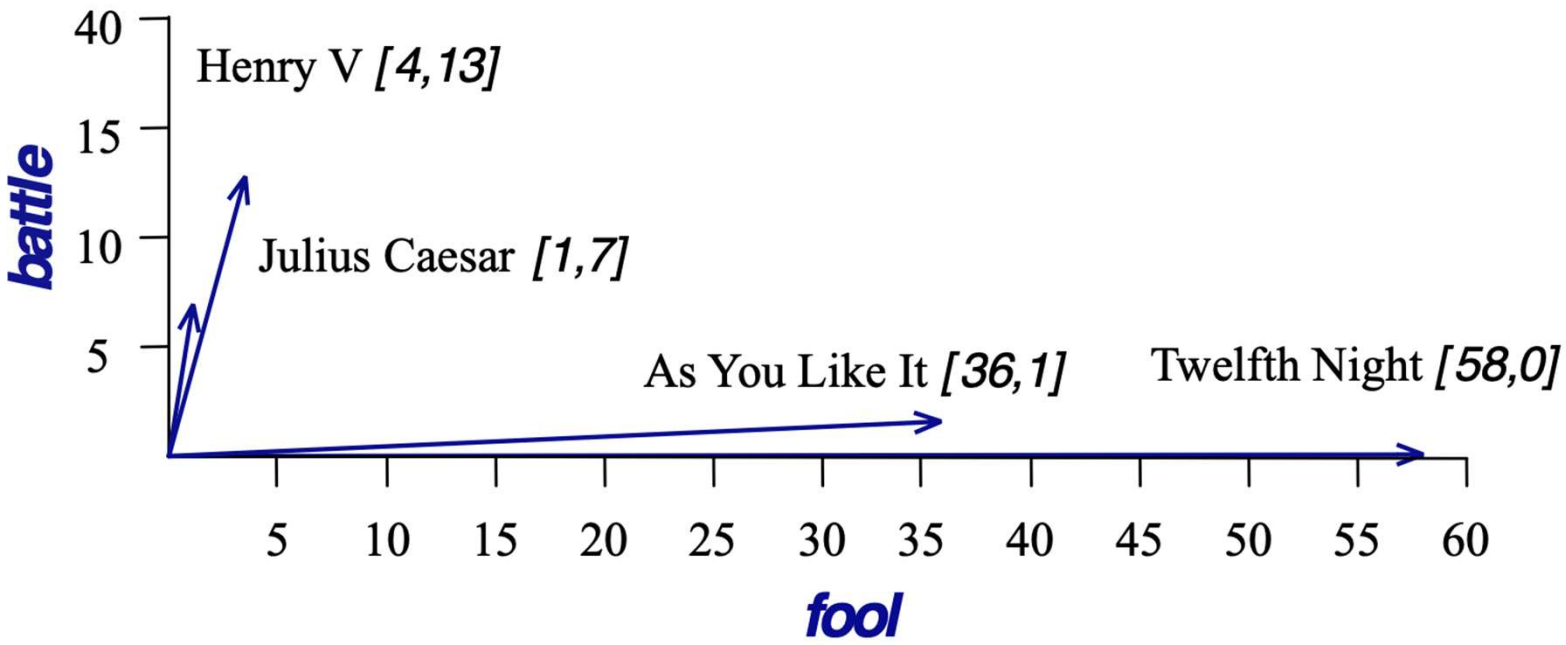
Example 1: Shakespeare plays
Each document is represented by a vector of words based on counts

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

vector

Visualizing document vectors

Dimensions: 2 words, (from the total of 4 dimensions)



Vectors are the basis of information retrieval

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Vectors are similar for the two comedies
Different than the history dramas

Comedies have more fools and wit and fewer battles.

Words can be vectors too

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

battle is "the kind of word that occurs in Julius Caesar and Henry V"

fool is "the kind of word that occurs in comedies, especially Twelfth Night"

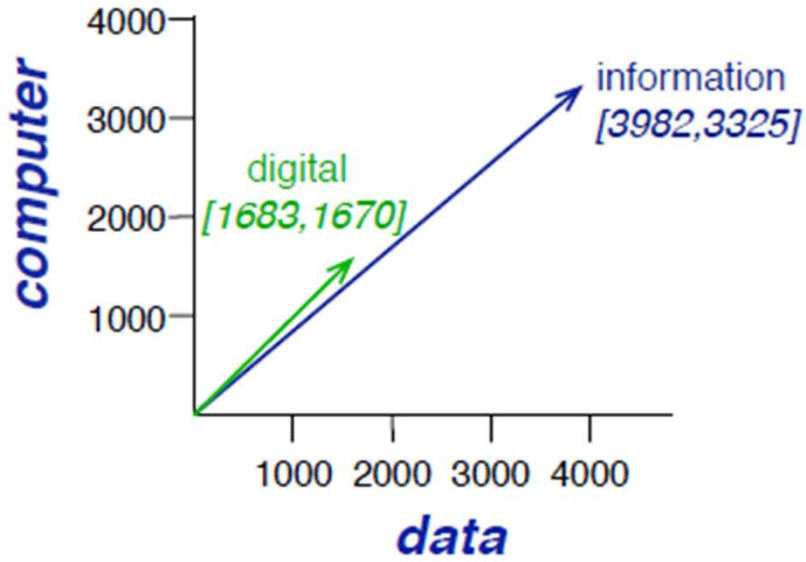
More common: word-word matrix (or "term-context matrix")

Two **words** are similar in meaning if their context vectors are similar

is traditionally followed by	**cherry**	pie, a traditional dessert
often mixed, such as	**strawberry**	rhubarb pie. Apple pie
computer peripherals and personal	**digital**	assistants. These devices usually
a computer. This includes	**information**	available on the internet

	a	**computer**	**pie**
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

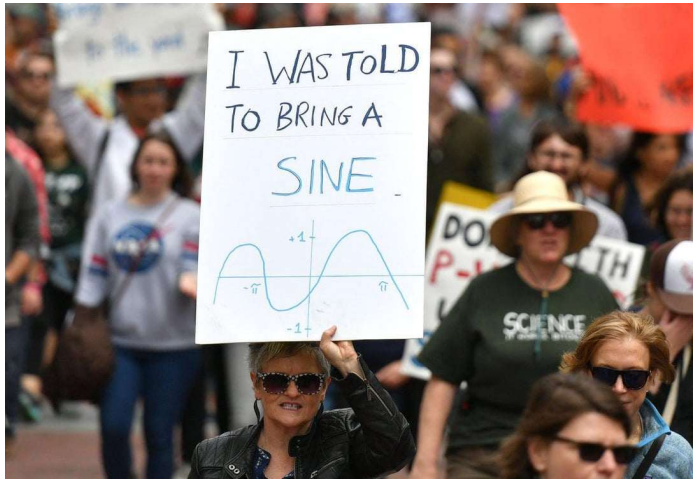
Word-word matrix visualised

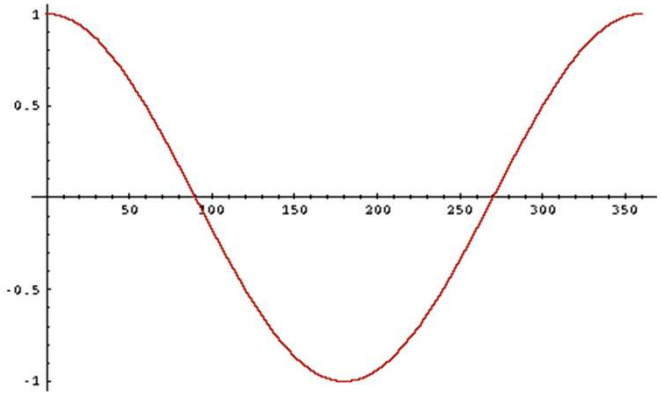


Many sentences later…

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Old friend: Cosine





Reminders from linear algebra

$$\text{dot-product}(\vec{v}, \vec{w}) = \vec{v} \cdot \vec{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + \ldots + v_N w_N$$

vector length $$|\vec{v}| = \sqrt{\sum_{i=1}^{N} v_i^2}$$

Cosine for computing similarity

Sec. 6.3

$$\text{cosine}(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}||\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

v_i is the count for word v in context i
w_i is the count for word w in context i.

$$\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| \cos\theta$$
$$\frac{\vec{a} \cdot \vec{b}}{|\vec{a}||\vec{b}|} = \cos\theta$$

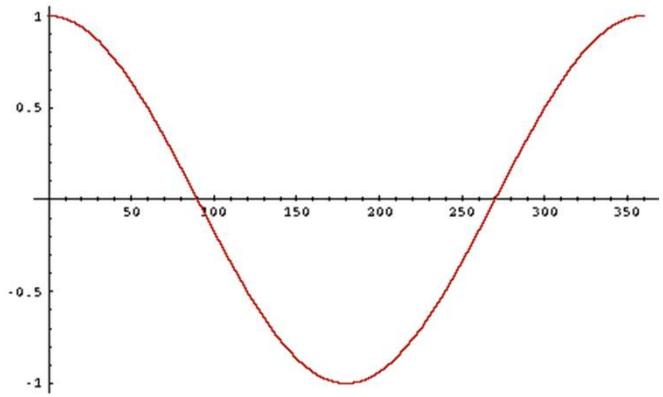
Cos($\vec{v}$,$\vec{w}$) is the cosine similarity of $\vec{v}$ and $\vec{w}$

Cosine as a similarity metric

-1: vectors point in opposite directions

+1: vectors point in same directions

0: vectors are orthogonal



Frequency is non-negative, so cosine range 0-1

Cosine similarity

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

Which pair of words is more similar?

cosine(cherry,information) =

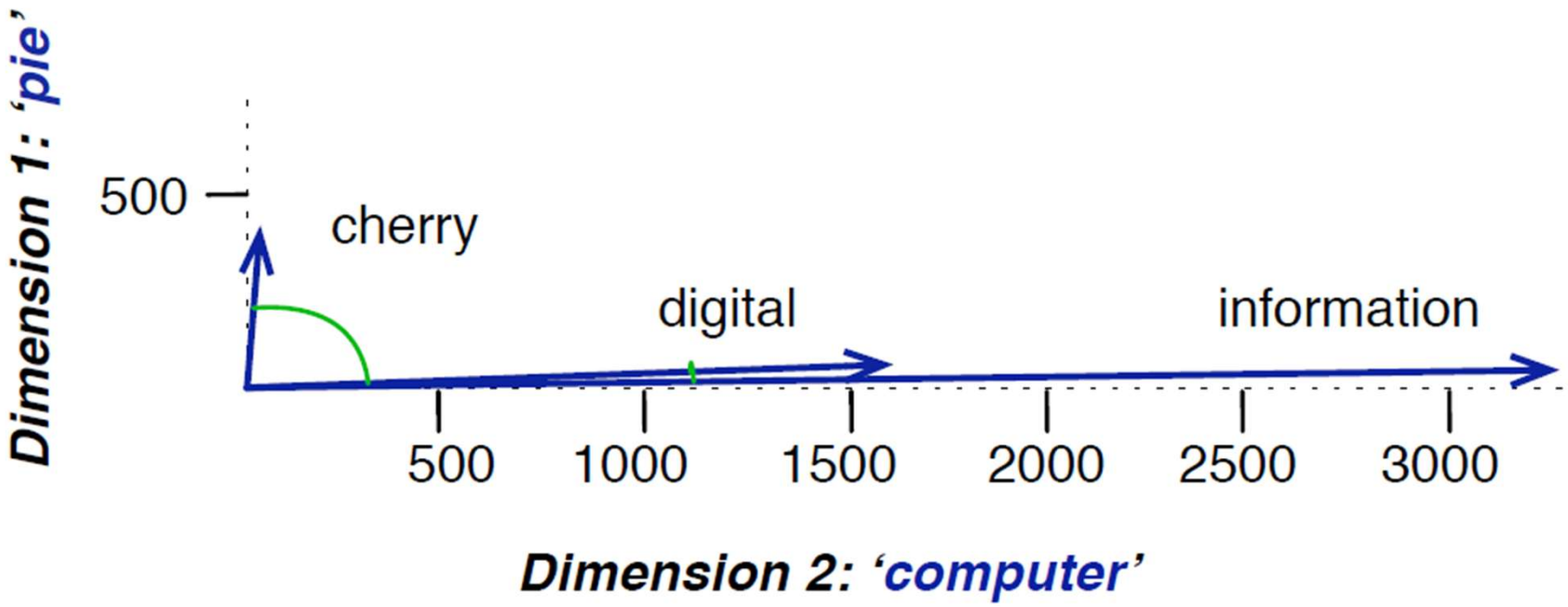
$$\frac{442*5+8*3982+2*3325}{\sqrt{442^2+8^2+2^2}\sqrt{5^2+3982^2+3325^2}}=.018$$

cosine(digital,information) =

$$\frac{5*5+1683*3982+1670*3325}{\sqrt{5^2+1683^2+1670^2}\sqrt{5^2+3982^2+3325^2}}=.996$$

$$\cos(\vec{v},\vec{w})=\frac{\vec{v}\bullet\vec{w}}{|\vec{v}||\vec{w}|}=\frac{\vec{v}}{|\vec{v}|}\bullet\frac{\vec{w}}{|\vec{w}|}=\frac{\sum_{i=1}^{N}v_iw_i}{\sqrt{\sum_{i=1}^{N}v_i^2}\sqrt{\sum_{i=1}^{N}w_i^2}}$$

Visualizing cosines in 2 dimensions
(well, angles)



But we get very sparce vectors

Vectors with as many dimensions as there are different words….

E.g.g Tf-idf and PPMI are sparse representations

tf-idf and PPMI vectors are

- **long** (length |V|= 20,000 to 50,000)
- **sparse** (most elements are zero)

Word2Vec - introduction

Problem with words as discrete symbols

Example: in web search, if user searches for "Seattle motel", we would like to match documents containing "Seattle hotel".

But:

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]
hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

These two vectors are orthogonal.

There is no natural notion of **similarity** for one-hot vectors!

Solution:

- Could try to rely on WordNet's list of synonyms to get similarity?
  - But it is well-known to fail badly: incompleteness, etc.
- **Instead: learn to encode similarity in the vectors themselves**

Alternative: dense vectors

vectors which are

- **short** (length 50-1000)
- **dense** (most elements are non-zero)

Work better for almost any NLP-task than sparce vectors

Sparse versus dense vectors

Why dense vectors?

- Short vectors may be easier to use as **features** in machine learning (less weights to tune)
- Dense vectors may **generalize** better than storing explicit counts
- They may do better at capturing synonymy:
  - *car* and *automobile* are synonyms; but are distinct dimensions
    - a word with *car* as a neighbor and a word with *automobile* as a neighbor should be similar, but aren't
- **In practice, they work better**

Dense embeddings you can download!

Word2vec (Mikolov et al. 2013)
https://code.google.com/archive/p/word2vec/

Fasttext http://www.fasttext.cc/

Glove (Pennington, Socher, Manning)
http://nlp.stanford.edu/projects/glove/

Intuition is the same

Representing words by their context

- Distributional semantics: **A word's meaning is given by the words that frequently appear close-by**
  - *"You shall know a word by the company it keeps"* (J. R. Firth 1957: 11)
  - One of the most successful ideas of modern statistical NLP!
- When a word *w* appears in a text, its **context** is the set of words that appear nearby (within a fixed-size window).
- Use the many contexts of *w* to build up a representation of *w*

…government debt problems turning into	***banking***	*crises as happened in 2009…*
…saying that Europe needs unified	***banking***	*regulation to replace the hodgepodge…*
…India has just given its	***banking***	*system a shot in the arm…*

These context words will represent ***banking***

Word vectors

We will build a dense vector for each word, chosen so that it is similar to vectors of words that appear in similar contexts

$$\textit{banking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}$$

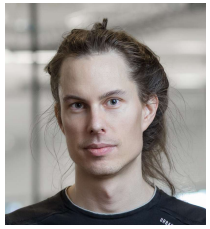
Note: word vectors are sometimes called word embeddings or word representations. They are a distributed representation.

Word2vec is self-supervision

Supervised – human labels define target (e.g. sentiments) in training data

Unsupervised – patterns in data and no human labels

Self-supervised – no human labels. Running text acts as implicitly supervised training data!



Word2vec: Framework for learning word vectors

Mikolov et al. 2013: Distributed representations of words and phrases and their compositionality.

- Large corpus
- Every word in fixed vocabulary is represented by **vector**
- Go through every position *t* in text, with centerword *w* and context words *c*
- Use similarity of Word vector for *w* and *c* to calculate ***P(c|w)*** **(skip-gram)** or *P(w|c)*(CBOW)
  - Instead of **counting** how often each word *w* occurs near "*apricot*"
  - Train a classifier on a binary **prediction** task: *Is w likely to show up near "apricot"?*
  - The learned classifier weights are the word embeddings
- Keep adjusting word vectors to maximize this probability

Word2Vec: Skip-Gram Task

Word2vec provides a variety of options. Let's do

- "skip-gram with negative sampling" (SGNS)
- CBOW Model

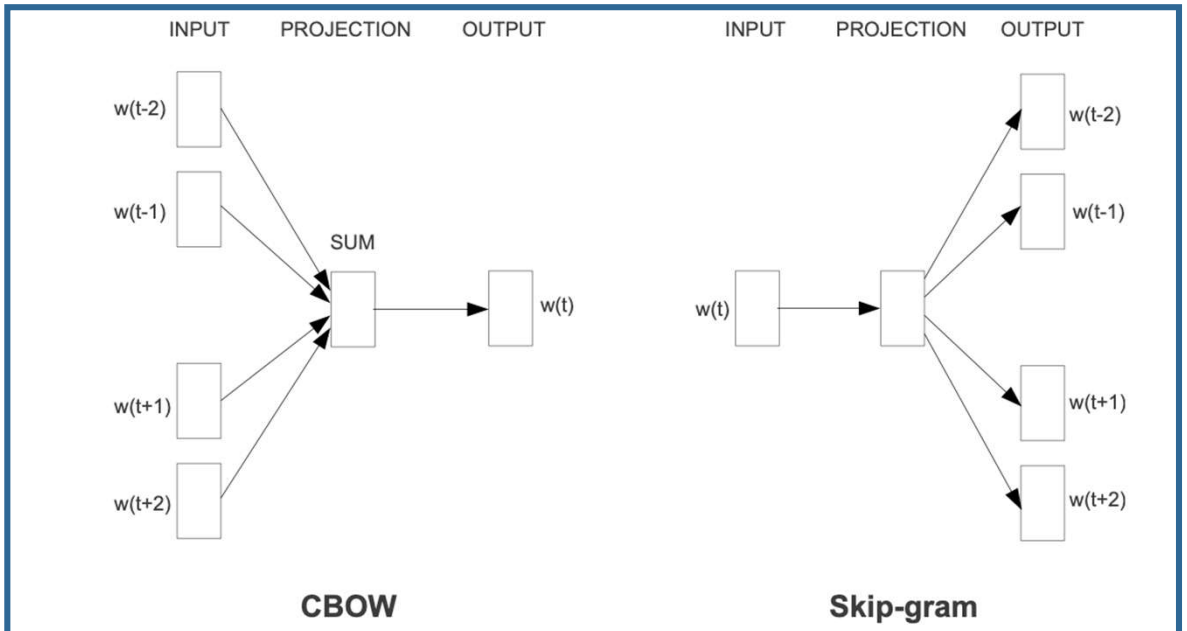


Figure 1: New model architectures. The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word.

https://arxiv.org/pdf/1301.3781.pdf

Symbols

w = center word
c = context word
L = window size
k = number of negative samples
+ = positive example
- = negative example

Skip-gram algorithm (SGNS)

1. Treat the target word and a neighboring context word as positive examples.
2. Randomly sample other words in the lexicon to get negative samples
3. Use sigmoid/logistic regression to train a classifier to distinguish those two cases
4. Use the weights as the embeddings

Skip-Gram Training Data

Training sentence:

... lemon, a tablespoon of **apricot** jam a pinch ...

c1 c2 target w c3 c4

Asssume context words are those in +/- 2 word window (L)

Skip-Gram Goal

Given a tuple (w,c) = target **w**ord, **c**ontext pair

- (*apricot*, *jam*)
- (*apricot*, *aardvark*)

Return probability that **c** is a real context word to w:

$P(+|w,c)$

$P(-|w,c) = 1-P(+|w,c)$

How to compute p(+|w,c)?

Intuition:

- Words are likely to appear near similar words
- Model similarity with dot-product (cosine)!
- Similarity(w,c) $\propto w \cdot c$
  - If $w \cdot c$ is large, words are similar / likely to occur together
  - If $w \cdot c$ is small or negative, words are less related

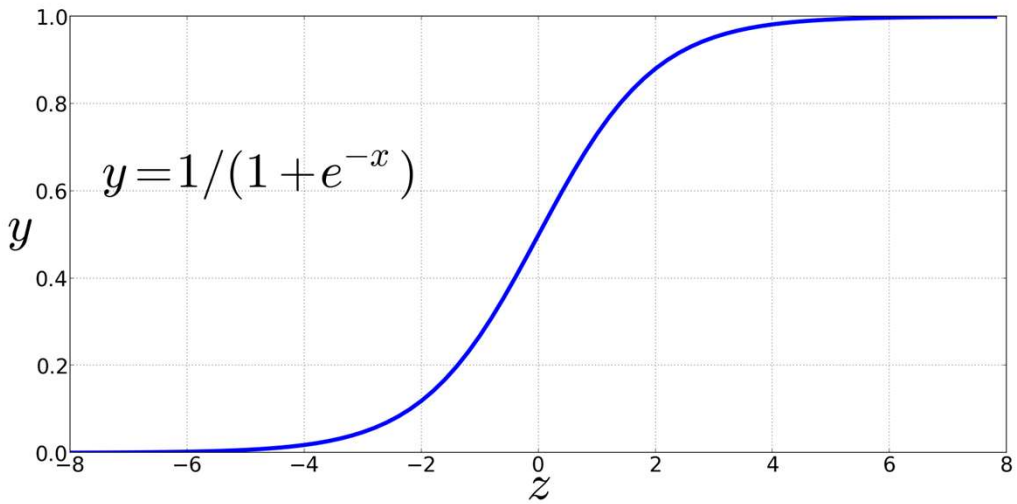
Problem:

- *We'll need to normalize to get a probability*

Turning dot product into a probability

The sigmoid function lies between 0 and 1:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



Just like logistic regression

Turning dot product into a similarity probability

Sim(w,c) ≈ w • *c*

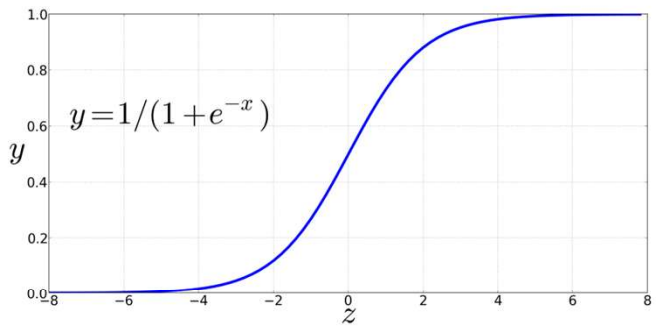
To turn this into a probability

We'll use the sigmoid from logistic regression:

$$P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1+\exp(-c \cdot w)}$$

$$P(-|w,c) = 1 - P(+|w,c)$$

$$= \sigma(-c \cdot w) = \frac{1}{1+\exp(c \cdot w)}$$



For all the context words:

Assume all context words are independent

$$P(+|w, c_{1:L}) = \prod_{i=1}^{L} \sigma(c_i \cdot w)$$

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^{L} \log \sigma(c_i \cdot w)$$

L= Context window

Back to Skip-Gram Training Data

Training sentence:

... lemon, a tablespoon of **apricot** jam a pinch ...

c1 c2 w c3 c4

Training data: input/output pairs centering on *apricot*

Asssume a +/- 2 word window (L)

Skip-Gram Training

Training sentence:

... lemon, a tablespoon of **apricot** jam a pinch ...

c1 c2 w c3 c4

positive examples +

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

- For each positive example, we'll create *k* negative examples.
- Using *noise* words
- Any random word that isn't *w*

Skip-Gram Training

Training sentence:

... lemon, a tablespoon of **apricot** jam a pinch ...

c1 c2 w c3 c4

k=2

positive examples +

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

w	c_{neg}	w	c_{neg}
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

Objective Criteria

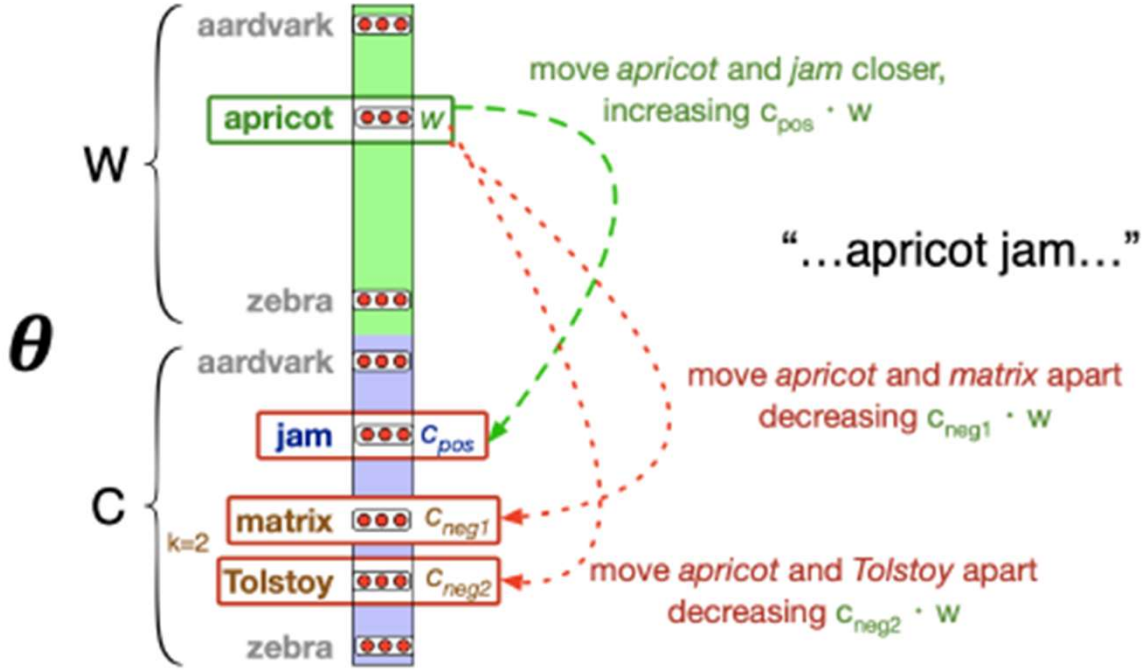
We want to maximize…

$$\sum_{(t,c)\in+} logP(+|t,c) + \sum_{(t,c)\in-} logP(-|t,c)$$

Maximize the similarity of the target word, context word pairs (w, c_{pos}) drawn from the positive data

Minimize the similarity of the (w, c_{neg}) pairs drawn from the negative data.

Learning the classifier: intuition



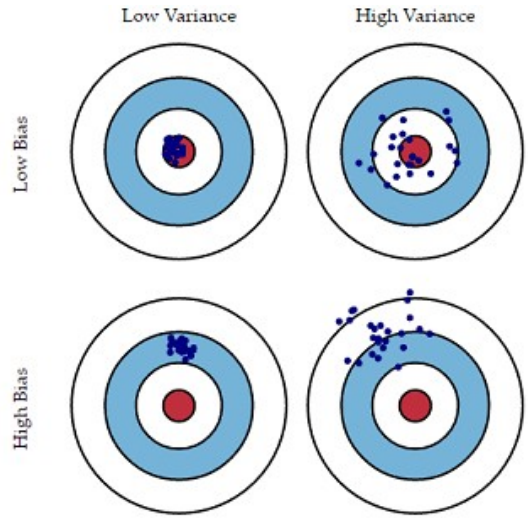
Summary: How to learn word2vec (skip-gram) embeddings

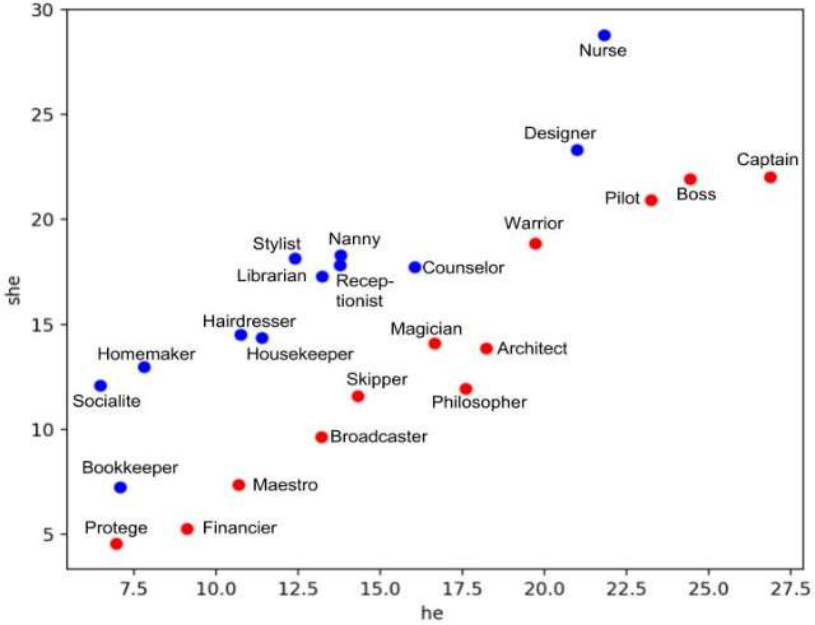
Start with V random d-dimensional vectors as initial embeddings

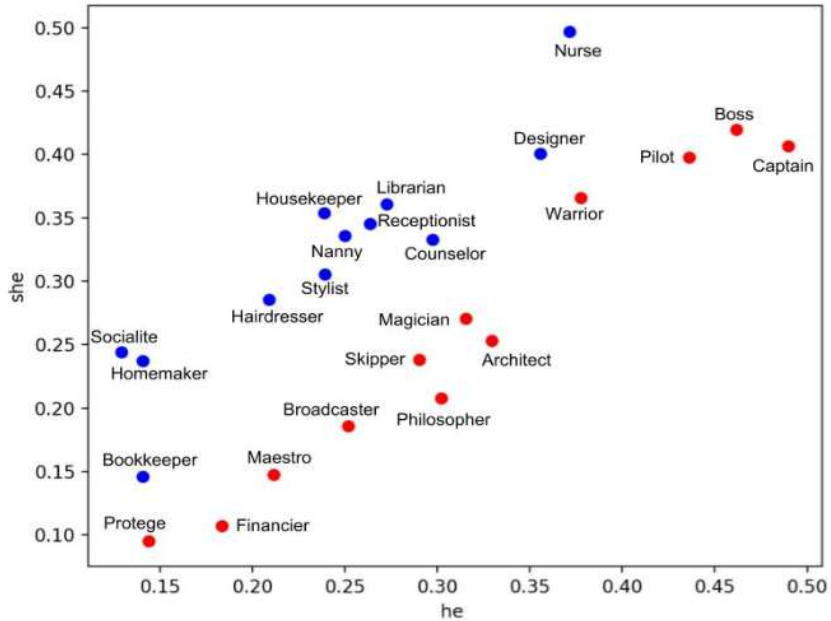
Train a classifier based on embedding similarity

- Take a corpus and take pairs of words that co-occur as positive examples
- Take pairs of words that don't co-occur as negative examples
- Train the classifier to distinguish these by slowly adjusting all the embeddings to improve the classifier performance
- Throw away the classifier code and keep the embeddings-weights.

Bias?


Low Variance
High Variance
Low Bias
High Bias


she
he
Nurse
Designer
Captain
Pilot
Boss
Warrior
Stylist
Nanny
Librarian
Recep-
tionist
Counselor
Hairdresser
Magician
Homemaker
Housekeeper
Architect
Skipper
Socialite
Philosopher
Broadcaster
Bookkeeper
Maestro
Protege
Financier


she
he
Nurse
Boss
Designer
Pilot
Captain
Housekeeper
Librarian
Receptionist
Warrior
Nanny
Counselor
Stylist
Hairdresser
Magician
Socialite
Skipper
Architect
Homemaker
Broadcaster
Philosopher
Bookkeeper
Maestro
Protege
Financier

How about Bias?

When the language model is self supervised we have no control over bias?

When a representation of the world is modelled, bias is also modelled (Weisenbaum 1976)

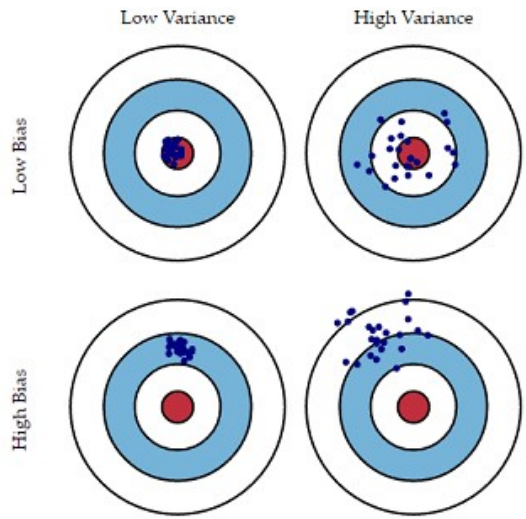
Examples of bias, that you have experienced (symbolic modelled, supervised or self-supervised)?

Problematic: Reinforces or amplifying stereotypes or unfair decisions

- hiring systems

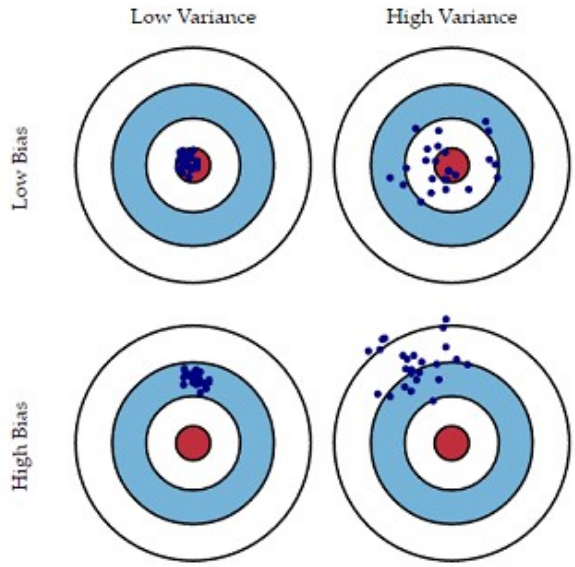
- recommendation systems

- search results



Bias in Word Embeddings

- Word embeddings are learned from real-world text (just like LLMs) and encode also stereotypes present in data
- The model has no understanding of fairness or ethics - reflects statistical regularities in the data
  - “doctor” closer to “man” than “woman”
  - “nurse” closer to “woman” than “man”
- Because:
  - Frequent co-occurrences → strong vector similarity
  - Self-supervised learning = no correction of bias
- How can we work with it in word embeddings?
  - Before training: Balance training data
  - After training: Penalties or adjustment of vectors
  - And much more…



Evaluation and implications of embeddings

Evaluating embeddings

Compare to human scores on word similarity-type tasks:

- WordSim-353 (Finkelstein et al., 2002)
- SimLex-999 (Hill et al., 2015)
- Stanford Contextual Word Similarity (SCWS) dataset (Huang et al., 2012)
- TOEFL dataset: *Levied is closest in meaning to: imposed, believed, requested, correlated*

Properties of embeddings

Similarity depends on window size L

L = ±2 The nearest words to *Hogwarts:*

- *Sunnydale*
- *Evernight*

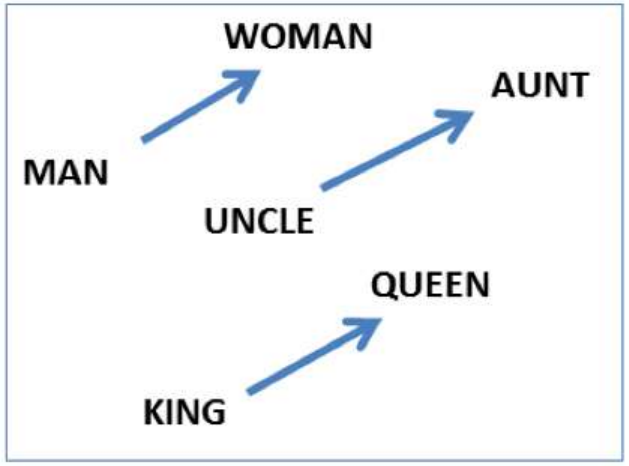
L = ±5 The nearest words to *Hogwarts:*

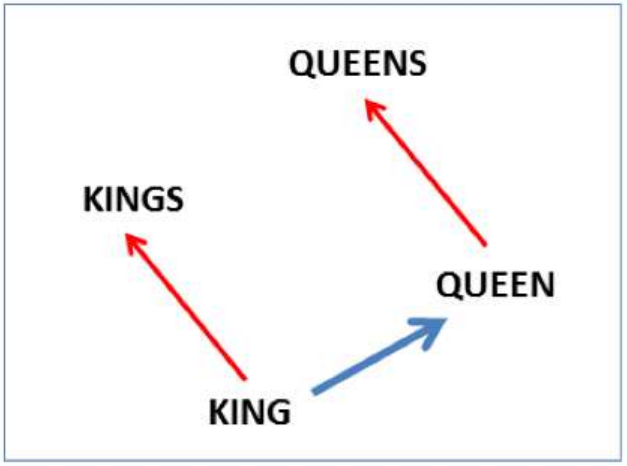
- *Dumbledore*
- *Malfoy*
- *halfblood*

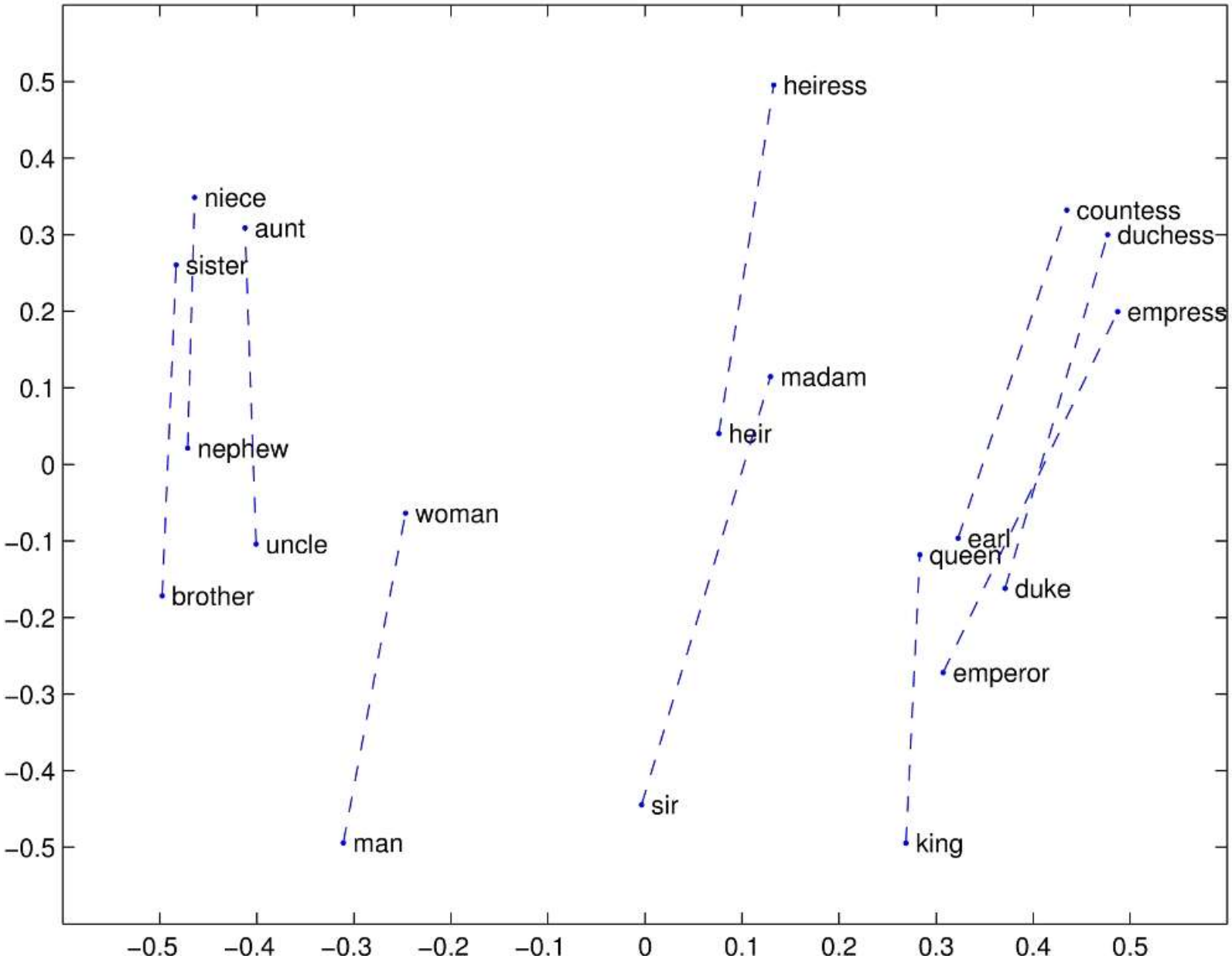
Analogy: Embeddings capture relational meaning!

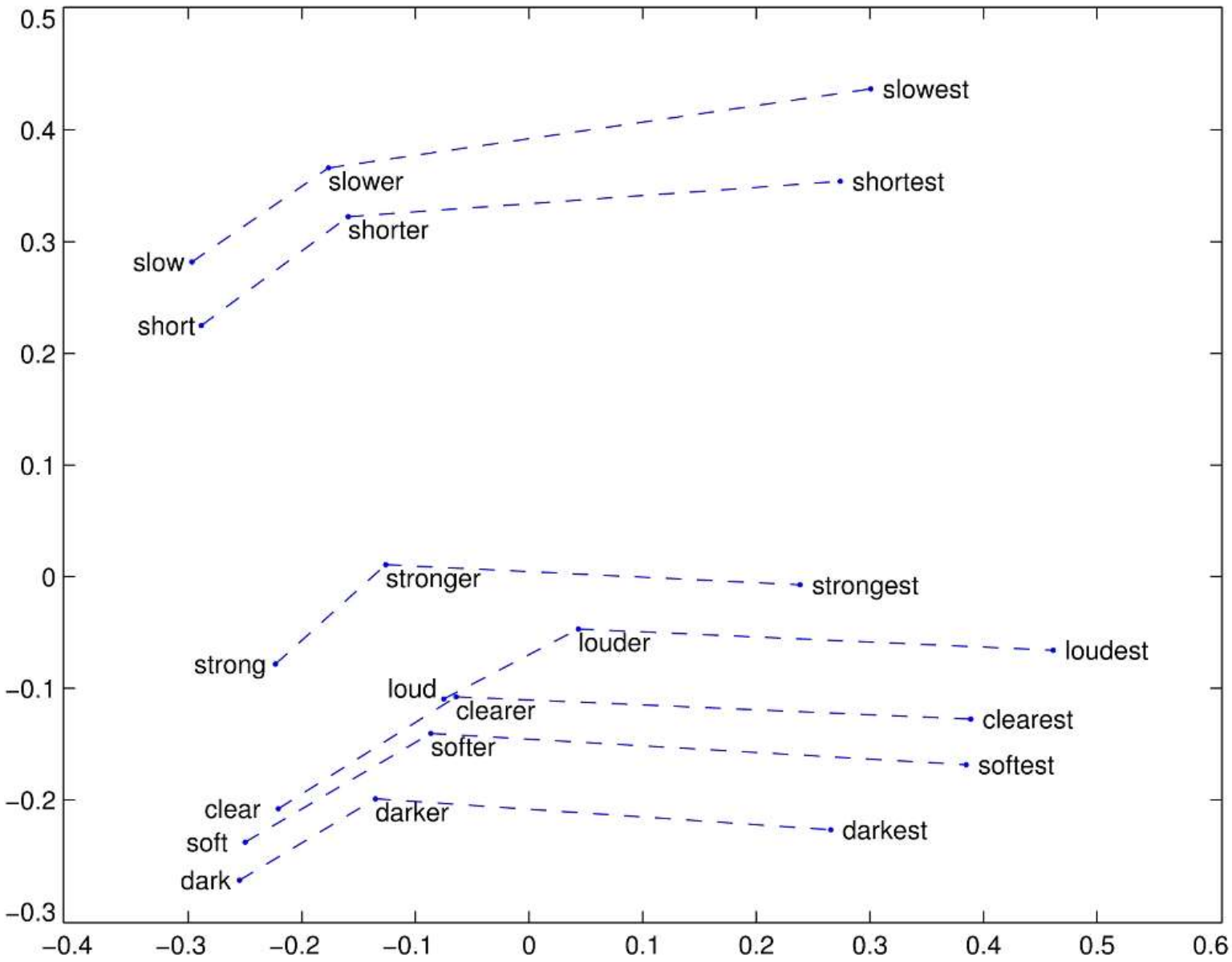
vector(*'king'*) - vector(*'man'*) + vector(*'woman'*) = vector('queen')

vector(*'Paris'*) - vector(*'France'*) + vector(*'Italy'*) = vector('Rome')






heiress
niece
aunt
sister
countess
duchess
empress
madam
heir
nephew
woman
uncle
earl
queen
duke
brother
emperor
sir
man
king
0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
−0.5
−0.5
−0.4
−0.3
−0.2
−0.1
0
0.1
0.2
0.3
0.4
0.5


0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
−0.3
−0.2
−0.1
0
0.1
0.2
0.3
0.4
0.5
0.6
slowest
slower
shortest
shorter
slow
short
stronger
strongest
louder
loudest
strong
loud
clearer
clearest
softer
softest
clear
darker
darkest
soft
dark